

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

Session 24

Architecture Clinic #2 + Phase 3 Wrap-Up

Reviewing Every Decision Since Session 1 • Phase 3 Complete • Phase 4 Capstone Preview

Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

SESSION 24

Architecture Clinic #2 + Phase 3 Wrap-Up

Reviewing Every Decision Since Session 1 · Phase 3 Complete · Phase 4 Capstone Preview

Purpose This session is not about adding another technology. It is about learning to judge the architecture you already built: what problem justified a decision, what trade-off it introduced, what evidence you have, what technical debt remains, and what you would deliberately change.

Student Tutorial

Use this document during the Architecture Clinic, group discussion, retrospective, and Capstone hand-off. Write your own reasoning. There is not one universal architecture that every team must choose.

1. Learning Objectives

By the end of Session 24, you should be able to:

- Evaluate architectural decisions using trade-offs rather than simply asking whether the system works.
- Explain why a technology or pattern was introduced at a particular point in the course.
- Distinguish a justified architectural decision from a premature or over-engineered decision.
- Identify technical debt that is resolved, still open, or newly discovered.
- Create Technical Debt Register v2 for the Phase 4 Capstone.
- Use evidence from previous sessions, tests, logs, load tests, and code to support an architecture discussion.
- Defend a decision while also explaining what evidence would make you reconsider it.
- Enter the Capstone phase with clear ownership of inherited architecture and explicit ownership of remaining design choices.

The Core Skill

Think like a senior engineer Do not stop at: “It works.” A stronger engineering statement is: “It works for this workload and requirement, we accepted these costs, and we would reconsider the decision if these conditions changed.”

2. What This Session Is Really About

Session 8 introduced the first Architecture Clinic. Session 24 repeats the exercise with much more context. By this point the platform has gone through observability, CQRS, real identity and OAuth2, service mesh, advanced reliability patterns, and performance testing. The supplied session plan explicitly frames the second clinic as a judgement exercise rather than a search for a single correct answer.

The most important question is therefore not:

“Which technology is best?”

Instead ask:

“For this problem, at this scale, with these constraints, what trade-off are we intentionally accepting?”

A Simple Decision Model

Step	Question to ask
1. Problem	What concrete problem or requirement are we solving?
2. Context	What scale, workload, team, constraints, and existing architecture matter?
3. Options	What realistic alternatives existed?
4. Trade-off	What did we gain, and what complexity or cost did we introduce?
5. Evidence	What tests, observations, failures, metrics, or code support the decision?
6. Reconsideration	What future condition would make us change the decision?

3. Phase 3 Recap — Sessions 17–23

Use this table as the starting point for the retrospective. The important exercise is not memorizing the technologies; it is connecting each addition to the problem that justified introducing it.

Session	What was added	Why it was introduced then
17 — Observability	Distributed tracing (Zipkin), metrics (Prometheus/Grafana), structured logs	Enough cross-service complexity existed to make tracing and operational visibility meaningful.
18 — CQRS	Command/query separation on product-service;	A concrete read/write conflict was described: dashboard-

	event-driven cache eviction	style reads versus checkout/write behavior.
19 — Security Part 1	Keycloak identity provider; Authorization Code flow	Answered the previously deferred question of where real user tokens come from.
20 — Security Part 2	Client Credentials flow; Gateway as OAuth2 Resource Server; service-to-service authentication	Completed the real security loop and retired the hand-rolled JWT validation approach.
21 — Service Mesh	Istio, mTLS, weighted traffic, mesh-level resilience	Addressed cross-cutting duplication and demonstrated mesh-level infrastructure capabilities.
22 — Advanced Patterns	Transactional Outbox, Idempotency Keys, API Versioning	Closed named technical-debt items related to retries/duplicates, API evolution, and reliable event publication.
23 — Performance	k6 load/stress testing and bottleneck/resilience analysis	Turned resilience configuration into something that could be exercised and observed under load.

Important The recap tells you what the course introduced. The clinic asks a harder question: would you make the same decision again, under the same context, and what evidence supports your answer?

4. Architecture Clinic Rules

During the discussion:

1. Do not defend a decision merely because it was taught or because it is common in industry.
2. Do not reject a technology merely because it adds complexity.
3. State the problem before defending the solution.
4. Name at least one benefit and one cost for important architectural decisions.
5. Use evidence where possible: test results, load-test observations, logs, traces, code complexity, operational steps, or failure behavior.
6. Distinguish “valuable to learn” from “necessary to deploy.” A technology can be educationally valuable while still being questionable for a small production system.
7. If you propose changing a decision, explain what would replace it and what new risks the replacement creates.
8. If you would keep a decision, explain what evidence justifies keeping it.
9. Do not claim that all technical debt is gone. A healthy system always has decisions that can be revisited.

5. The Five Architecture Questions

Work in small groups. Spend approximately eight minutes on each question. Rotate the facilitator so that every participant practices architectural reasoning.

Question 1 — Did CQRS earn its complexity?

Review the Session 18 product-service CQRS split.

Discuss: Discuss whether the described dashboard-versus-checkout conflict justified the additional model and flow.

Avoid: Do not answer with “CQRS is good” or “CQRS is over-engineering.” Explain the actual problem, scale, benefit, and cost.

Useful evidence: Evidence to seek: read/write paths, model separation, query behavior, complexity added to code and operations.

Group notes:

Question 2 — Was Istio the right call for this platform?

Review Session 21.

Discuss: Separate the value of learning service-mesh concepts from the question of whether this particular platform requires a mesh.

Avoid: Discuss sidecar/control-plane cost, learning and operational complexity, and the cross-cutting problems the mesh addresses.

Useful evidence: Evidence to seek: number of services, operational overhead, configuration complexity, mTLS/traffic-management needs, resilience duplication.

Group notes:

Question 3 — Was retiring hand-rolled JWT validation a clean cutover?

Review Sessions 19–20.

Discuss: Discuss the move to Keycloak/OAuth2 and retirement of the earlier JwtAuthFilter.

Avoid: For a controlled training environment, a clean cutover may be reasonable. For a live system with external consumers, migration and compatibility concerns could change the decision.

Useful evidence: Evidence to seek: token issuer, audience/claims, gateway behavior, service-to-service flow, client migration requirements.

Group notes:

Question 4 — What debt is still open?

Compare the earlier Technical Debt Register with the work completed by Session 22.

Discuss: Identify specific debt that remains and decide whether it belongs in Phase 4.

Avoid: Do not say “everything is resolved.” Name a concrete item and explain its priority.

Useful evidence: Candidate areas: persistent Saga state, database migrations, dead-letter replay/alerting, older hardcoded route/config patterns.

Group notes:

Question 5 — What ONE earlier decision would you reconsider?

Imagine you are rebuilding the platform today with all the knowledge gained through Session 23.

Discuss: Choose one decision from Sessions 1–16 and explain what you would change.

Avoid: A strong answer identifies a concrete change, explains the original trade-off, and describes the cost/risk of changing it.

Useful evidence: Possible discussion areas include when observability should begin, how resilience and messaging decisions evolved, and when reliability patterns become necessary.

Group notes:

6. How to Give a Strong Architecture Answer

Use the following structure when presenting your group answer:

Part	Example structure
Decision	“We chose X for Y.”
Problem	“The concrete problem was...”
Benefit	“The main benefit was...”
Cost	“The complexity/cost introduced was...”
Evidence	“Our evidence is...”
Alternative	“An alternative would have been...”
Reconsideration	“We would change this if...”
Conclusion	“Given the current context, we would keep/change/defer it because...”

Example of reasoning Instead of “Istio is industry standard,” a stronger answer discusses the cross-cutting problem it solves, the operational overhead it introduces, the scale of the platform, and the conditions under which the team would or would not adopt it.

7. Technical Debt Register v2

Technical debt is not simply “bad code.” It is a known limitation, shortcut, missing capability, or architectural compromise that may create future cost or risk. The purpose of Register v2 is to make those decisions visible to the Capstone team.

7.1 Resolved Items to Verify

- ☐ Retry without idempotency on payment calls → Idempotency Keys (Session 22).
- ☐ No API versioning strategy → /api/v1/ plus the defined deprecation approach (Session 22).
- ☐ Hardcoded JWT secret / no real identity provider → Keycloak + OAuth2 (Sessions 19–20).

☐ No structured logging → structured JSON logs + trace correlation (Session 17).

7.2 Open Items to Carry Into Phase 4

☐ In-memory SagaState in OrderSagaOrchestrator — state can be lost on restart.

☐ No database migration tooling — schema changes have been managed manually.

☐ Dead-letter queue handling — events are logged, but automated replay/alerting remains immature.

☐ Hardcoded configuration patterns such as PUBLIC_ROUTES may remain in older session code.

☐ Any additional debt discovered during today's clinic.

7.3 Student Technical Debt Register

ID	Debt / limitation	Evidence	Impact	Priority	Proposed action / owner
TD-01					
TD-02					
TD-03					
TD-04					
TD-05					
TD-06					

How to Prioritize Debt

Factor	Ask
Impact	What happens if we leave it unresolved?
Likelihood	How likely is the problem to occur?
Scope	Does it affect one service or the whole platform?
Cost of delay	Will fixing it later become substantially harder?
Capstone dependency	Will new Capstone features depend on this area?
Learning value	Does resolving it demonstrate an important engineering capability?

8. Architecture Decision Record (ADR) Mini-Template

When your group identifies an important decision, capture it as a short ADR. This makes the reasoning reusable during the Capstone.

Field	Write this
Title	Short decision name
Status	Accepted / Reconsider / Deferred
Context	What problem and constraints existed?
Decision	What are we choosing?
Alternatives	What realistic alternatives were considered?
Benefits	What do we gain?
Costs / risks	What complexity, cost, or failure modes do we accept?
Evidence	What observations or tests support the decision?
Reconsider when	What future condition would trigger a review?

9. Phase 4 Capstone Preview

Phase 4 begins with the platform you already built. The goal is not to blindly preserve every previous decision and not to rewrite everything. The team must understand what is inherited, what is open, and what it is allowed to change.

9.1 Inherited Foundation

- Config Server / Eureka / Gateway foundations from Sessions 1–3.
- Resilience4j stack from Sessions 4–5.
- OpenFeign communication from Session 6.
- Choreography Saga from Session 7.
- Docker, Kubernetes, CI/CD and GitOps work from Sessions 9–16.
- Observability foundation from Session 17.
- Security stack from Sessions 19–20.

9.2 Decisions the Capstone Team Must Own

- Whether CQRS should be introduced in additional areas.
- Whether the Service Mesh should cover more services or remain partial.
- Which open technical-debt items deserve priority.
- Whether a new Capstone feature creates new architectural requirements.
- Whether an inherited decision should be retained, modified, or replaced—and why.

Important distinction Inherited does not mean “untouchable.” It means the team starts from an established baseline. Any change should be justified by a concrete requirement, problem, evidence, or risk.

10. Capstone Architecture Review Checklist

Before leaving Session 24, your team should be able to answer all of these:

- ☐ Can we explain the purpose of every major Phase 3 addition?
- ☐ Can we name at least one trade-off for CQRS?
- ☐ Can we name at least one trade-off for Istio/service mesh?
- ☐ Can we explain why Keycloak/OAuth2 replaced the earlier authentication approach?
- ☐ Can we identify the remaining technical debt?
- ☐ Can we distinguish resolved debt from deferred debt?
- ☐ Have we recorded any new debt discovered during the clinic?
- ☐ Do we know which inherited components we are expected to keep working?
- ☐ Do we know which architectural decisions are explicitly open to Capstone judgement?
- ☐ Can every team member explain at least one architectural decision without reading the slides?
- ☐ Can we state what evidence would cause us to reconsider a decision?

11. Individual Reflection

Complete this individually after the group clinic.

The architectural decision I understand differently now is:

The decision I would reconsider first is:

The most important trade-off I learned to recognize is:

A piece of technical debt I believe the Capstone should address is:

A decision I would keep—and the evidence supporting it—is:

One condition that would make me change my mind is:

12. Final Student Deliverables

Your team should leave Session 24 with:

- 10. Completed discussion notes for the five Architecture Clinic questions.
- 11. Technical Debt Register v2, including resolved, open, and newly surfaced items.
- 12. At least one short ADR for a decision your team debated seriously.
- 13. A clear list of inherited platform components.
- 14. A clear list of architectural choices the Capstone team owns.
- 15. A prioritized list of technical-debt candidates for Phase 4.
- 16. An individual reflection from each team member.

Minimum Quality Standard

Weak answer	Stronger answer
“It works.”	“It works for the current context; here is the evidence and trade-off.”

“X is industry standard.”	“X solves this concrete problem; these are its costs for our context.”
“We should use it everywhere.”	“We should use it where this problem exists and justify each extension.”
“All debt is resolved.”	“These items are resolved; these remain open; these are newly discovered.”
“I would change everything.”	“I would change this specific decision because of this evidence or trade-off.”
“I would change nothing.”	“I reviewed the alternatives and identified why the current choices still fit.”

13. Session 24 Takeaway

Architecture is not a collection of technologies.

Architecture is a set of decisions, trade-offs, evidence, and consequences.

The Phase 3 clinic is complete when you can explain not only what the platform contains, but why it contains it, what you would question, what debt remains, and what you are deliberately choosing as a Capstone team.

Next Phase 4 begins with the supervised Capstone Build Day. Bring Technical Debt Register v2, your architecture notes, and your decision rationale with you.

Appendix A — One-Page Architecture Review Worksheet

Decision	Notes
What problem were we solving?	
What context mattered?	
What alternatives existed?	
What did we gain?	
What did we add in complexity/cost?	
What evidence do we have?	
What would make us reconsider?	

Appendix B — Capstone Handoff Summary

Area	Current state	Keep / Change / Defer	Reason
Infrastructure / discovery			
Gateway / routing			
Resilience			
Communication / Saga			
Observability			
Security			
CQRS / read models			
Service mesh			
Messaging reliability			
API evolution			
Performance			
Technical debt			